Final Project Report - Generating Product Images from Customer Reviews

Sri Bhamidipati - Shriya Jejurkar - Jason Liu - Kaitlin Moore - Lindberg Simpson
https://github.com/kaitlinmoore/review-driven-product-image-generation

---

Executive Summary

This project investigates whether generative AI can reconstruct what a product looks like from product metadata and customer review text alone. We built a 5-agent pipeline that ingests Amazon reviews, narrows them down into structured visual descriptions, and drives two-image generation models (FLUX.1-schnell and gpt-image-1.5) towards photorealistic product imagery. This system's unique contribution is a Surrogate-Projected Adaptive Iteration Control loop, where a Controller agent fits quadratic surrogate models to the run's own execution history, thus reallocating compute between attempts based on observed difficulty, without prior knowledge of which products are the difficult ones.

We evaluated 4 products, a chess set, water bottle, jeans, and backpack, over a 4 x 2 x 3 design (24 independent runs overall), with 3 independent embedding encoders and a 13-field structured-feature scoring framework. The results classify cleanly by review category: when reviewers write visually, the pipeline reproduces the product near ceiling (chess set, 33% visual pass rate), and when they write about durability, it has no real signal to refine itself on (backpack, 4%). The strongest finding we had was that more sophisticated prompting may actually hurt products like jeans, because reviewers describe fit and functionality over appearance. This finding challenges the implicit assumption that richer inputs always produce richer outputs.

Q1. Product Selection and Customer Review Data Collection

Data Source

All review data was drawn from the UCSD Amazon Reviews 2023 corpus maintained by the McAuley Lab (mcauleylab.ucsd.edu). The dataset spans 33 product categories and contains millions of records per category, distributed as gzipped JSONL files with one review per line and one file per category. Rather than downloading complete files, our pipeline streams and decompresses each file using Python's requests library with chunked HTTP responses. This bypasses the HuggingFace Parquet mirror, which exposes only 9 of the 33 categories. This streaming approach keeps memory usage bounded regardless of category size and enables us to scan over 169 million review rows in total across the four products.

Alongside reviews, we collected each product's metadata: title, store, rating distribution, image URLs, the bullet-point feature list, and the prose description text from the Amazon listing. These are stored

per-product at data/{slug}/metadata.json. The metadata feeds into the synthesis stage (Q2) alongside the top-ranked visual reviews.

Product Selection Methodology

Product selection was designed to maximize experimental contrast across three axes: product category, visual complexity, and review language type. We evaluated over 40 candidate products across five Amazon categories and four popularity bands (by total rating count), then applied a four-criterion filter: appropriate rating-count band, keyword match to a specific canonical ASIN, at least one usable product image, and non-trivial product description text. While popularity was one of several selection axes we considered, the visual signal strength present in the actual number of customer reviews (often a fraction of the total rating count) proved more valuable and predictive in our exploratory analysis. The final selection therefore optimizes visual signal diversity, including four products that deliberately span a wide range of expected difficulty for the pipeline.

Product	Category	Visual Challenge	Selection Rationale
Chess Set	Toys & Games	Intricate piece geometry, wood grain, carved detail	● High rating count (6,824) ● Enthusiasts describe physical appearance in detail ● strong visual vocabulary
Water Bottle	Home & Kitchen	Cylindrical form, brand text rendering, specific colorway	● Very high rating count (55,702) ● Bright fuchsia color is distinctive ● Tests brand-text generation
Jeans	Clothing, Shoes & Jewelry	Garment fit and texture, on-model composition, seam details	● High rating count (10,581) ● Reviewers describe fit and feel rather than appearance ● Tests schema-forcing
Backpack	Sports & Outdoors	Straps, compartments, rainfly, technical features	● High rating count (9,599) ● Reviewers focus on durability, not looks ● Intended baseline/limit case

Table 1. Final product selection spanning four Amazon categories with graduated visual challenge. "Rating count" includes all Amazon customer ratings, many of which do not include text, but the number is nonetheless an indicator of product popularity.

Exploratory Data Analysis: Review Statistics and Visual Gate

The "Raw Reviews" count in Table 2 (below) differs from the Amazon ratings counts in Table 1 because the two measure different things. Amazon's total ratings count (Table 1) includes both star-only ratings (no text) and reviews-with-text. The McAuley UCSD 2023 corpus we used (Table 2) contains only reviews-with-text, since the dataset is built for text analysis. The gap between the two (typically 60–90% of Amazon's total ratings) represents customers who rated but did not write. Table 2's counts are what the pipeline actually processed.

After keyword-matching and ASIN filtering, we applied a visual content gate through a GPT-4o-mini classifier that reads each review and decides whether it describes how the product looks. The table below presents the resulting pass rates and the review counts available for the agent's refinement loop. The 8× spread in pass rate between the chess set (33%) and the backpack (4%) is an interesting finding of the EDA phase as it could be an early indicator of the quality ceiling for each product before the pipeline runs a single image.

Product	Avg. Rating	Raw Reviews	Visual Pass Rate	Avail. for Refinement	Rows Scanned
Chess Set	4.7	1,535	~33%	477	16.3M
Water Bottle	4.8	9,313	~17%	1,529	67.4M
Jeans	4.3	2,363	~12%	244	66.0M
Backpack	4.5	813	~4%	0 (limit case)	19.6M

Table 2. Review collection statistics and visual-content filter pass rates. After removing the top-30 reviews used to seed the initial prompt, the backpack has zero remaining visual reviews for the agent loop. "Raw Reviews" is the count of written reviews actually present in our scraped corpus and matching the canonical ASIN.

The chess set's 33% pass rate reflects a community of hobbyists who photograph their boards, describe wood grain, and discuss piece craftsmanship in tactile, visual language. Backpack reviewers, by contrast, evaluate weight, durability, frame stiffness, and zipper reliability, which are features that are experiential rather than visible. Jeans reviewers occupy a middle position as they describe fit and feel accurately but rarely describe seam color, stitching pattern, or exact wash. This category-level signal heterogeneity is a structural property of the Amazon review ecosystem, not a sampling artifact, and it has direct implications for any review-driven generative pipeline.

Q2. Analysis of Customer Reviews with LLM

Visual Content Filtering Gate

The first processing stage applies a zero-shot classification pass using GPT-4o-mini to every review in the matched set. The classifier is asked to give structured output with a pass/fail rating and reason determined by whether the review describes how the product looks, as opposed to how it performs, feels, or compares in price. Reviews classified as non-visual are discarded before any downstream processing. This gate is intentionally conservative as a false negative (discarding a visual review) is less harmful than a false positive (feeding non-visual language into a prompt that drives an image model). The gate operates review-by-review with no batching, because visual language tends to cluster within individual sentences rather than at the review level. A reviewer who devotes 80% of their text to assembly instructions and 20% to the wood finish is treated as a visual reviewer. The 20% visual sentences are preserved for downstream use.

Although a vector database is a natural consideration for storing text embeddings, our pipeline's requirements made a lazy PyTorch DataLoader a more appropriate choice. Vector databases are useful when the core operation is semantic retrieval, but our loop does not retrieve in that sense. The visual content gate pre-filters and helpfulness-ranks all candidate reviews before the agent begins, so what the loop needs is budget-gated sequential consumption, not similarity search. The DataLoader's iterator pattern supports this natively, while also keeping memory bounded across 169 million streamed review rows. An upfront vector ingestion step would have imposed significant infrastructure overhead for a corpus consumed in a fixed ranked order. The one compromised capability is query-conditioned retrieval within the loop, for instance allowing the Refiner to surface color-specific reviews when the current prompt is weak on that field, and we treat this as a direction for future work.

Structured Feature Schema

Downstream scoring requires a common schema that can represent both what reviewers wrote and what the generated image shows. We designed a 13-field schema (Table 3) with three field types and corresponding scoring functions. The schema is applied identically to:

(a) the review corpus, producing a "reviews" feature vector
(b) the ground-truth product photograph, via GPT-4o vision
(c) each generated image, also via GPT-4o vision.

This triple application makes every comparison field-level and interpretable rather than a single opaque similarity score.

Field	Scoring Type	Description
primary_colors	Set | Jaccard	Color vocabulary (e.g., brown, fuchsia, navy)
materials	Set | Jaccard	Physical material (wood, stainless steel, denim, nylon)
decorative_elements	Set | Jaccard	Ornamentation (carvings, stitching, inlays)
visible_parts	Set | Jaccard	Components visible in the image (board, pieces, straps, cap)
size_descriptor	Enum | exact match	Perceived size category (large, medium, small)
brand_visibility	Enum | exact match	Whether branding is present and legible
finish	Text | token-set Jaccard	Surface finish (matte, gloss, natural, brushed)
texture	Text | token-set Jaccard	Tactile or visual surface texture (smooth, grainy, worn-in)
shape_and_form	Text | token-set Jaccard	Overall product shape (cylindrical, square-board, bootcut)
brand_description	Text | token-set Jaccard	Textual brand description (HYDRO CELL logo, etc.)
overall_aesthetic	Text | token-set Jaccard	Holistic style descriptor (classic, rugged, sleek, elegant)
product_type	Identifier only	Product category label (reference, not scored)
measurements	Identifier only	Stated dimensions (reference, not scored)

Table 3. The 13-field structured feature schema. Each of the 11 scored fields contributes 1/11 to the composite feature agreement score.

The chess set provides the clearest illustration of the schema's diagnostic power. Reviewers describe wood species with precision, for example they mention beech, birch, hornbeam, and sycamore. However, the ground-truth image call returns only "wood." Reviewers cite board dimensions (21.65 inches square), while the image call returns null. This mismatch reflects a genuine asymmetry between what customers attend to when they write versus what a camera captures when it photographs. The schema surfaces these gaps as named field mismatches rather than as an undifferentiated quality deficit.

Prompt Engineering Strategy

After filtering and ranking, the top-30 highest-scoring visual reviews and product metadata are passed in a single batch to GPT-4o with a structured prompt instructing the model to use metadata as the factual foundation and reviews to add aesthetic nuance the listing doesn't capture. The output, initial_prompt.txt, becomes the seed for image generation across all configurations. The same GPT-4o call also extracts the initial structured feature vector. This design holds the prompt-construction pipeline constant across all runs, making the v1→v2→v3 ablation a controlled comparison of the in-loop scoring rule alone.

Three configurations were designed to isolate the contribution of different scoring rules, while holding the agent architecture, the initial prompt, and the inner-loop refinement mechanism constant across all runs. The image generation prompt in every configuration is the converged output of the Mistral Refiner, derived from initial_prompt.txt and the consumed review batches. What changes across configurations is the in-loop quality signal (the rule the Evaluator uses to score generated images), which then feeds the Controller's retune.

v1 | Title-CLIP: The in-loop quality signal is CLIP cosine similarity between the generated image and the product title string, with a quality target of $q^* = 0.5$. The title is the shortest possible specification of what the product is, and v1 measures how well the agent can converge while being told only whether the result matches a category-level label.

v2 | Prompt-CLIP: The in-loop quality signal is CLIP cosine similarity between the generated image and the full initial_prompt.txt text, a ~250-word visual synthesis produced by GPT-4o from the top-30 visual reviews and product metadata. The metric is unchanged from v1; only the scoring reference is richer. This isolates the effect of giving CLIP a more detailed reference text to compare against.

v3 | Feature-Agreement: The in-loop quality signal switches metrics. The Evaluator extracts the 13-field structured-feature schema from the generated image (via GPT-4o vision) and computes per-field agreement against the schema extracted from initial_prompt.txt. The quality target rises to $q^* = 0.70$ to reflect the more discriminating natural scale of feature agreement (CLIP cosines top out around 0.40 for product photos. Feature agreement lives in the 0.4–0.7 range for unrefined images). This is the most structured and computationally expensive configuration. Each in-loop scoring call requires a GPT-4o vision request, but it produces the most interpretable feedback for the Controller.

The v1 → v2 comparison isolates the scoring reference (same metric, richer reference text). The v2 → v3 comparison isolates the scoring metric (same reference, holistic CLIP cosine vs. per-field structured agreement). Each step changes one variable.

Q3. Image Generation with Diffusion Model

Experiment Design

The image generation experiment uses a fully crossed 4 × 2 × 3 design as we had four products, two image generation models (FLUX.1-schnell and gpt-image-1.5), and three prompt configurations (v1, v2, v3), for 24 independent runs in total. Each run produces three image attempts via the agent loop. The two models represent different priors and cost profiles:

FLUX.1-schnell: An open-weights diffusion model noted for texture and material rendering quality. It is substantially faster and cheaper to run than proprietary alternatives, making it practical for the agent's multi-attempt control loop. Its primary weaknesses are brand-text rendering and unusual compositional priors.

gpt-image-1.5: OpenAI's proprietary image generation model. Its training distribution gives it stronger studio-composition priors and more reliable brand-text rendering, at higher per-image cost. It is the preferred model when identity-level fidelity and legible branding are the primary targets.

Evaluation Framework

The evaluation framework uses three distinct evaluation families to ensure that findings are not artifacts of any single metric choice:

Family A - In-loop trajectory:

The agent's own quality signal during refinement. For v1 and v2 this is CLIP cosine similarity between the generated image and the text reference (title or prompt). For v3 it is structured-feature agreement against the initial prompt's feature vector. These values drive the control loop and are reported as in_loop_best_q in the agent metadata. They are not used as the final judgment of output quality, as these are done in Families B and C.

Family B - Post-hoc image-vs-image cosine similarity:

Three independent embedding models, CLIP, DINOv2, and SigLIP, each compute cosine similarity between the final generated image and the real product photograph. Using three encoders trained on different objectives (caption alignment, self-supervised structure, vision-language fine-tuning) ensures that directional agreement across all three constitutes a robust finding rather than an artifact of any single metric's biases.

Family C - Post-hoc structured-feature agreement vs. ground truth:

The 13-field structured schema is extracted from the generated image via GPT-4o vision and compared field-by-field against the same schema extracted from the ground-truth product photograph. Set fields (colors, materials, visible parts) are scored by Jaccard overlap; enum fields (size, brand visibility) by exact match; free-text fields (finish, texture, overall aesthetic) by token-set Jaccard. Each of the 11

scored fields contributes 1/11 to the composite score, yielding a per-field failure taxonomy in addition to a single interpretable number.

The three embedding models differ in their training objectives in a way that makes their disagreements informative. CLIP is trained on image-caption pairs optimized for broad semantic alignment. DINOv2 is self-supervised on visual structure and is highly sensitive to identity-level differences. SigLIP is a recent vision-language model with improved fine-grained alignment. When all three agree, the result is robust. When they disagree, the direction of disagreement reveals something specific about the nature of the pipeline's success or failure.

Per-Product Results

Chess Set

The chess set is the experiment's headline result. All three embedding encoders score the best generated images near ceiling (Table 4), and all three agree directionally. Rich review language of features like wood species names, board dimensions, piece silhouettes and felt lining, translates cleanly into diffusion model prompts because the vocabulary is precise and visual. The structured-feature agreement score against the ground truth reaches 0.81 with FLUX v3, the highest value observed across all products, and 0.74 with gpt-image v3.

Config	In-Loop Best Q	CLIP Mean	DINOv2 Mean	SigLIP Mean	Feat. Agr. GT
FLUX.1-schnell					
v1 | Title-CLIP	0.324	0.867	0.741	0.839	0.730
v2 | Prompt-CLIP	0.319	0.867	0.741	0.839	0.730
v3 | Features	0.429	0.830	0.728	0.826	0.812
gpt-image-1.5					
v1 | Title-CLIP	0.324	0.864	0.734	0.856	0.728
v2 | Prompt-CLIP	0.324	0.864	0.734	0.856	0.728
v3 | Features	0.423	0.861	0.741	0.851	0.737

Table 4. Chess set evaluation results. Bold values indicate best per-column per-model. CLIP/DINOv2/SigLIP are image-vs-ground-truth cosine similarity (mean over 3 attempts). Feat. Agr. GT = structured-feature agreement vs. ground truth.

The pipeline accurately generated the wood board with visible grain, carved Staunton-style pieces with distinct knight silhouettes, felt base and storage compartments, and the overall natural-wood aesthetic. However it did not get the decorative inlay border around the board perimeter, since it was described

only qualitatively by reviewers as "elegant trim". It also struggled with the exact piece proportions (king height, base flare), and the interior compartment configuration. These misses are consistent as they correspond to features that reviewers describe through adjectives rather than geometric specifics. The structured-feature v3 configuration achieves the highest feature agreement with the ground truth for FLUX (0.81), confirming that more structured prompting helps when the review corpus provides structural vocabulary.

Water Bottle

The water bottle produces the most analytically interesting result in the experiment as CLIP and DINOv2 disagree by 30 points on the same image pair. CLIP scores the gpt-image v1/v2/v3 outputs at a mean of 0.741, while DINOv2 scores the same images at 0.436. SigLIP ranks the highest at 0.743. This 30-point gap is the expected consequence of evaluating with encoders trained on different objectives.

Config	In-Loop Best Q	CLIP Mean	DINOv2 Mean	SigLIP Mean	Feat. Agr. GT
FLUX.1-schnell					
v1 — Title-CLIP	0.277	0.754	0.413	0.660	0.579
v2 — Prompt-CLIP	0.280	0.689	0.420	0.613	0.636
v3 — Features	0.524	0.689	0.420	0.613	0.636
gpt-image-1.5					
v1 — Title-CLIP	0.306	0.741	0.436	0.743	0.786
v2 — Prompt-CLIP	0.312	0.741	0.436	0.743	0.786
v3 — Features	0.488	0.741	0.436	0.743	0.786

Table 5. Water bottle evaluation results. CLIP and DINOv2 disagree by ~30 points, representing qualitatively different notions of "closeness."

CLIP focuses on whether this is a fuchsia stainless-steel water bottle, while the generated images are, so CLIP scores 0.74. However, DINOv2 focuses on whether this is specifically the HYDRO CELL 24oz with this cap geometry and seam profile, and the generated images are not, so DINOv2 scores 0.44. Both answers are correct but they address different questions. The structural shape (cylindrical), the dominant color (fuchsia), and the material (matte stainless finish) all render correctly. The brand text on the generated bottle is illegible gibberish, the specific cap geometry does not match, and the seam details are absent. This proves that the evaluator you choose determines what "closeness" means and is a choice that must be made consciously and defended.

Jeans

The jeans experiment produces the sharpest analytical finding. v3 fails to improve over v2 in either model, and on FLUX it produces the same final image as v2 (0.497 vs 0.497) despite the agent's in-loop quality signal rising substantially. On GPT, v3 actively underperforms v2 (0.571 vs 0.617). And on FLUX, even v1, the simplest configuration, outperforms v3 (0.694 vs 0.497). This counter-intuitive result is consistent with the EDA finding that jeans reviewers describe fit and feel rather than visual structure.

Config	In-Loop Best Q	CLIP Mean	DINOv2 Mean	SigLIP Mean	Feat. Agr. GT
FLUX.1-schnell					
v1 — Title-CLIP	0.282	0.895	0.480	0.760	0.694
v2 — Prompt-CLIP	0.326	0.853	0.439	0.742	0.497
v3 — Features *	0.503	0.853	0.439	0.742	0.497
gpt-image-1.5					
v1 — Title-CLIP	0.294	0.882	0.557	0.786	0.594
v2 — Prompt-CLIP	0.330	0.889	0.584	0.791	0.617
v3 — Features *	0.450	0.873	0.547	0.751	0.571

*Table 6. Jeans evaluation results. * indicates configs where v3 underperforms v1 on Feat. Agr. GT despite higher in-loop quality signal.*

The v3 in-loop quality signal rises to 0.503 (FLUX) and 0.450 (GPT) as the agent believes it is making progress. However, the post-hoc ground-truth feature agreement drops relative to v1: from 0.694 to 0.497 for FLUX and from 0.594 to 0.571 for GPT. This divergence occurs because the 13-field schema being optimized in-loop was extracted from review text that does not meaningfully describe visual structure. The agent is optimizing against a reference that is underspecified on visual fields, and the schema's best-response is to hallucinate field values not grounded in the reviews. Thus, producing images that score well on the synthetic in-loop metric but worse on the real-world ground truth. The holistic CLIP scores remain high (0.853–0.895) because the generated images are recognizably jeans but DINOv2 drops to 0.439–0.584 because they are not specifically these jeans.

Backpack

The backpack is kept in the experiment as a deliberate limit case. With only 4% of reviews passing the visual content gate, and the top 30 used to seed the initial prompt, the agent has zero visual reviews remaining for its refinement loop. All three configurations produce identical outputs because the agent's DataLoader is exhausted before the first refinement step. The per-image iteration counts in the agent metadata confirm this: [0, 0, 0] across all six runs (Table 7).

Config	Iterations [0,0,0]	CLIP Mean	DINOv2 Mean	SigLIP Mean	Feat. Agr. GT
FLUX.1-schnell					
v1 — Title-CLIP	✓ [0,0,0]	0.813	0.551	0.649	0.156
v2 — Prompt-CLIP	✓ [0,0,0]	0.813	0.551	0.649	0.156
v3 — Features	✓ [0,0,0]	0.813	0.551	0.649	0.156
gpt-image-1.5					
v1 — Title-CLIP	✓ [0,0,0]	0.824	0.633	0.729	0.555
v2 — Prompt-CLIP	✓ [0,0,0]	0.824	0.633	0.729	0.555
v3 — Features	✓ [0,0,0]	0.824	0.633	0.729	0.555

Table 7. Backpack evaluation results. All six runs take zero refinement iterations. Within each model, all three configurations produce identical images and identical scores. The stark difference in Feat. Agr. GT between FLUX (0.156) and GPT (0.555) reflects the models' different generative priors for hiking gear, not review signal.

The backpack results contain one additional finding as despite no review-driven refinement, the two image models produce very different structured-feature agreement scores against the ground truth: 0.156 for FLUX versus 0.555 for GPT. Because both models receive exactly the same prompt and perform zero refinement steps, this gap is entirely attributable to model priors. gpt-image-1.5 has evidently seen substantially more hiking gear in training and is better positioned to supply the visual details that reviews never described. This represents the model's baseline knowledge about the product category acting as a substitute for absent review signal, a useful but uncredited contribution to output quality in no-signal situations.

Prompt Ablation: v1 vs. v2 vs. v3

Across four products and two models, the relationship between prompt strategy and image quality is not monotonic. Table 8 summarizes the structured-feature agreement against the ground truth for both models, highlighting which configuration wins per product.

Product	FLUX v1	FLUX v2	FLUX v3	GPT v1	GPT v2	GPT v3	Best Config
Chess Set	0.730	0.730	0.812	0.728	0.728	0.737	v3 ✓
Water Bottle	0.579	0.636	0.636	0.786	0.786	0.786	v2/v3
Jeans	0.694	0.497	0.497	0.594	0.617	0.571	v1/v2 ⚠
Backpack	0.156	0.156	0.156	0.555	0.555	0.555	Flat (0 iters)

Table 8. Structured-feature agreement against ground truth by product, model, and prompt strategy. ⚠ = v3 degrades quality vs. simpler strategies.

v3 wins convincingly for the chess set as the structured schema operationalizes the visual vocabulary that chess reviewers actually provided. For the water bottle, v2 and v3 are equivalent as the richer prompt text (v2) captures sufficient detail that the feature schema refinement in v3 adds no marginal value. For jeans, v1 is the best FLUX configuration and v2 the best GPT configuration, meaning that simpler, less structured strategies preserve the coarse visual signal better than a schema that forces the model to optimize fields the reviews never filled. For the backpack, all configurations are equivalent because the agent never executes a single refinement step.

Model Comparison: FLUX.1-schnell vs. gpt-image-1.5

Averaging CLIP image-vs-ground-truth scores across all three prompt strategies per product, gpt-image-1.5 leads FLUX.1-schnell on most products (Table 9). The largest gap appears on the water bottle and backpack. On the chess set, the models are essentially tied at the CLIP level, though gpt-image leads on SigLIP (0.851–0.856 vs. 0.826–0.839). On jeans, gpt-image consistently outperforms FLUX on DINOv2 (0.563 vs. 0.453 averaged), suggesting stronger identity-level visual discrimination.

Product	FLUX CLIP Avg.	GPT CLIP Avg.	FLUX DINOv2 Avg.	GPT DINOv2 Avg.
Chess Set	0.855	0.863	0.737	0.737
Water Bottle	0.711	0.741	0.417	0.436
Jeans	0.867	0.881	0.453	0.563
Backpack	0.813	0.824	0.551	0.633

Table 9. CLIP and DINOv2 similarity scores averaged across v1/v2/v3 per product.

gpt-image-1.5's main advantages are consistent studio-composition priors and better brand-text rendering. FLUX.1-schnell's main advantages are cost (significantly cheaper per image) and texture-rendering quality, where it is competitive on products with strong material vocabulary (chess set, backpack). For production use, the optimal model choice is product-dependent: gpt-image-1.5 is preferable when identity-level fidelity and brand-text legibility are required, while FLUX.1-schnell is preferable when material texture is the primary visual target and cost is a constraint.

Q4. AI Agentic Workflow

The system below qualifies as agentic on 3 counts: it consumes data as a function of decision made at runtime, it composes multiple LLM calls inside an inner control loop with non-trivial termination conditions, and it includes a controller that observes its own execution history and rewrites its own parameters between attempts. The first two are properties of any iterative LLM pipeline, and the third provides a feedback edge from the controller back into the refiner.

System Architecture

The pipeline is divided into two phases that differ fundamentally in their computational character. The preprocessing phase consists of one-shot, deterministic, non-iterating steps: the visual content gate, deduplication, helpfulness ranking, and a single GPT-4o synthesis call that produces `initial_prompt.txt` and the initial feature vector.

The agent phase begins when preprocessing ends. Five agents operate over shared memory to adaptively refine the generation prompt and allocate compute:

Refiner (Mistral-7B): Pulls a batch of visual reviews from the lazy PyTorch DataLoader and rewrites the current generation prompt to incorporate the new visual evidence. The DataLoader is an iterator, not a pre-loaded list. Reviews are consumed one batch at a time, only when the Controller's current budget authorizes another inner-loop step. This lazy consumption is the most concrete evidence that the system is not a fixed pipeline: a run on the chess set, where reviews are visually rich and the descriptiveness threshold fires quickly, consumes far fewer review tokens than a run on the water bottle, where the Rater repeatedly fails to clear the threshold and the budget cap fires instead. The Refiner shares its Mistral-7B instance with the Rater and forms the tight inner refinement loop with it.

Rater (Mistral-7B): Scores the Refiner's rewritten prompt on a descriptiveness scale of 0 to 100, asking: how well does this prompt describe the visual appearance of the product? The Rater and Refiner alternate in the inner loop — rewrite, rate, rewrite, rate — until one of two termination conditions fires: either the prompt's descriptiveness score clears the Controller's current threshold, meaning the prompt is visual enough to proceed to image generation, or the per-image iteration budget set by the Controller is exhausted. On easy products (chess set), the threshold fires in as few as 3 iterations. On hard products (water bottle), the inner-loop budget cap fires because the review batches cannot drive descriptiveness high enough to clear it — a signal that the Controller uses to recalibrate.

Generator (FLUX.1-schnell or gpt-image-1.5): Receives the Refiner's converged prompt and renders a single image. One Generator call occurs per outer iteration. The Generator is the only agent that

produces output observable by humans; all other agents produce intermediate representations (rewritten prompts, descriptiveness scores, quality scalars, updated control parameters) that are consumed entirely within the pipeline. Because the Generator call is the most expensive step in wall-clock time and API cost, the Controller's budget management directly controls total generation cost: fewer outer iterations on easy products means fewer Generator calls.

Evaluator (CLIP cosine or structured-feature agreement): Scores the generated image against the configured reference and returns a scalar quality value q_i. In v1 and v2 configurations, the Evaluator computes CLIP cosine similarity between the generated image embedding and the text embedding of the title (v1) or the full initial prompt (v2), with a target of $q^* = 0.5$. In v3, the Evaluator extracts the full 13-field structured-feature schema from the generated image via a GPT-4o vision call and computes field-level agreement against the initial prompt's feature vector, with a target of $q^* = 0.70$. The scalar q_i, together with the descriptiveness d and cumulative iteration count t observed during this outer iteration, is appended to the shared trajectory buffer — the only write operation that crosses the boundary from the generation side of the system to the Controller.

Controller (the novel contribution): Reads the trajectory buffer — three parallel tensors of descriptiveness d, cumulative iterations t, and quality q, one row per completed outer iteration — and fits two independent quadratic surrogate models: one mapping d to expected quality, one mapping t to expected quality. It then inverts each surrogate for the quality target q^* using the quadratic formula, projects the solution onto the feasible non-negative set, and writes two updated control parameters back into the Refiner: a new descriptiveness threshold and a new per-image iteration budget. The feedback edge from the Controller back to the Refiner is the one edge that a fixed pipeline would not have. It is what allows the system to allocate proportionally more compute to harder products — higher threshold, larger budget — without being told in advance which product is hard. To handle the early-trajectory case where fewer than three observations make the 3×3 design matrix rank-deficient, a synthetic origin point (0, 0) is prepended before fitting, ensuring the system is fully determined after just two real outer iterations.

Control Loop Algorithm

The outer loop generates image attempts sequentially. For each attempt, the inner loop refines the prompt until the Rater clears the Controller's current descriptiveness threshold, or until the per-image budget fires, whichever comes first. After the Evaluator scores the resulting image, the Controller returns its parameters for the next attempt. The trajectory buffer grows by one row per outer iteration.

Three design choices deserve explicit justification. First, the inner loop has two termination conditions: on easy products (chess set), the descriptiveness threshold fires quickly and the controller learns to allocate short budgets. On hard products (jeans, water bottle), the budget fires because the Rater cannot clear the threshold, and the controller learns to increase it. Second, the DataLoader is a lazy iterator: the most direct evidence that the system is not a pipeline. Reviews are consumed only as the controller authorizes additional refinement, meaning that a run on the chess set consumes far fewer tokens than a run on the water bottle. Third, the synthetic origin point (0, 0) is prepended to the trajectory before the surrogate fit, ensuring the 3×3 design matrix is fully determined after just two real observations.

Mathematical Framework and Convergence

The Controller fits two independent quadratic surrogate models, one in descriptiveness d, one in cumulative iterations t, to the observed quality values in the trajectory buffer. Each surrogate maps a scalar control variable to expected quality, using ordinary least squares with Moore-Penrose pseudoinverse for numerical stability when the trajectory is short. The quality target q^* (0.5 for CLIP-based configs, 0.70 for feature-based configs) is then inverted through each surrogate using the quadratic formula, projecting onto the feasible non-negative set and choosing the root closest to the current state.

Because the iteration variable t is cumulative, the Controller's update is an increment Δt rather than an absolute budget. Under the reasonable assumption that quality is a concave function of refinement effort (diminishing returns), the gap between the current quality and the target shrinks monotonically as the agent settles rather than oscillates. This convergence behavior is precisely what allows the system to allocate proportionally more compute to hard products without prior knowledge of difficulty. The agent observes its own execution history, infers the shape of the quality-vs-effort curve, and projects where it needs to be to hit the target.

Reproducibility

Every external API call and heavy local model inference in this pipeline is wrapped in a content-addressed cache (replay.py's cached_call). Each call's inputs (model name, temperature, system and user prompts, and SHA-256 hashes of any image inputs) are deterministically hashed. The resulting cache entry is keyed by that hash. On subsequent runs with identical inputs, the cache hit returns the prior output without making a live call.

The cache supports three modes via the REPLAY_MODE environment variable:

1. record
   Default mode that will make live calls if input does not match prior entries; will return cached entries if match is found
2. replay
   Returns cached entries when inputs are matched; raises error if match is not found
3. force_record
   Forces live call (and will thus overwrite cached entries)

The committed replay_cache/ directory contains every entry produced in our runs. System-prompt SHA-256 hashes are baked into cache-key inputs, so any edit to a prompt automatically invalidates affected entries. This prevents silent drift between code and cached results.

All results can be reproduced offline by cloning the repository, setting REPLAY_MODE=replay, and rerunning src/run_post_hoc_eval.py. No API keys or GPU are needed. Additional detailed instructions and documentation can be found in the REPRODUCIBILITY.md file in the repo root.

Analytics and Discussion

The experimental results show that outcome quality is governed primarily by:

1) the visual density of the review corpus
2) the alignment between scoring metric and what the corpus actually contains
3) the choice of the image model.

Why the chess set worked:
The synthesis prompt combined reviewer vocabulary ('Staunton-style' pieces, 'hand-burned flourishes') with metadata-sourced detail ('hornbeam', 'sycamore', '21.65 inches square') to produce a description rich in precise visual nouns. When the Refiner ingests this language, the Rater's descriptiveness signal rises quickly, and the Generator receives a prompt whose tokens map cleanly to diffusion-model vocabulary. The structured-feature schema's per-field design lets v3 specifically recover from cases where reviewers and the ground-truth photo describe overlapping but non-identical attributes, which means reviewers list 4 wood species while the photo just shows "wood." That overlap is what makes the v3 strategy win convincingly here (FLUX 0.81, GPT 0.74).

Why CLIP and DINOv2 disagreed on the water bottle:
CLIP scored the gpt-image v3 outputs at 0.74 while DINOv2 scored the same images at 0.44, proving a 30-point gap on identical pixels. CLIP, trained on caption alignment, asks "is this a fuchsia stainless-steel water bottle?" and answers yes. DINOv2, trained on self-supervised structural features, asks "is this the HYDRO CELL 24oz with this seam profile?" and answers 'no'. Both are correct evaluations of different questions. The implication for any review-driven image pipeline is that the choice of evaluator implicitly defines the success criterion, and that choice deserves to be made consciously rather than defaulted to.

Why v3 hurt the jeans:
This is the most analytically valuable result because it is counter-intuitive. The structured-feature strategy actively degraded ground-truth feature agreement from 0.69 (v1) to 0.50 (v3) on FLUX. The mechanism is that the schema being optimized in-loop was extracted from review text that does not describe visual structure. For example, jeans reviewers write about waistband drape and worn-in feel, not seam geometry. When the Controller raised its descriptiveness threshold and the Refiner consumed more reviews searching for visual content, it instead pulled adjective-level fit cues that the schema then forced into structural fields. The agent's quality signal rose because the schema looked complete; the post-hoc metric fell because the completion was hallucinated. This is a pattern any structured-prompting pipeline can fall into when the structure forces fields the source data cannot fill.

Why the backpack confirmed the mechanism:
With zero reviews remaining for the agent loop after the top-30 seed, the system reduces to a single non-iterating generation pass. All three configurations produce identical outputs, and the gap between FLUX (0.16) and GPT (0.56) on ground-truth feature agreement is entirely attributable to model priors learned during pre-training. This isolation, although unintended, demonstrates that a substantial fraction of generative output quality on niche product categories comes from the model's general visual prior rather than from any pipeline-specific signal, a useful but rarely-credited contribution.

Across the full prompt ablation, no single configuration dominates. v3 wins for the chess set (visual reviews), v2 ties for the water bottle (rich prompt sufficient), v1 or v2 wins for jeans (less structure preserves coarser signal), and v3 cannot run for the backpack. This non-monotonicity is a robust experimental finding rather than methodological noise: the optimal prompting strategy is contingent on the linguistic character of the review corpus, not on a fixed ordering of "simple to complex."

Insights

1. **Product category, not pipeline design, sets the achievable quality ceiling:**
   Before the LLM runs, before any prompt is crafted, the review corpus already determines the outcome. The visual content gate's pass rate varies 8× between the chess set (33%) and the backpack (4%), and this single statistic is a dominant predictor of pipeline behavior. Any team building a similar pipeline should treat the EDA-stage visual density measurement as a go/no-go signal, not a curiosity. Categories below ~10% visual pass rate are not realistic targets for review-driven generation.
2. **Structured prompting is non-monotonic in quality:**
   The intuition that more structure helps was disconfirmed for two of four products. The schema-forcing v3 strategy improved chess set output by 8 points but degraded jeans output by 20 points, both in the same metric, on the same code path. The mechanism is precise: structure helps when the structure matches the natural distribution of source language, and hurts when it forces the source to fill fields it does not contain. This argues for product-class-specific schemas rather than one universal schema, and for treating "richer prompt" as a hypothesis to test rather than a default to apply.
3. **"Closeness" is a metric choice, not a fact about the image:**
   The water bottle result establishes that two reasonable evaluators can disagree by 30 points on the same pair of images. CLIP measures category-level alignment; DINOv2 measures identity-level structure. Neither is wrong. Pipelines that report a single similarity number without naming the encoder should be read as making an unstated commitment to one definition of success. Triangulating with three encoders (CLIP, DINOv2, SigLIP) provides directional robustness but also surfaces these disagreements as informative signals rather than noise.
4. **Failure modes are predictable from the review corpus:**
   The pipeline reliably gets product type, primary colors, materials, and overall aesthetic right across all four products. It reliably misses brand text, fine decorative elements, exact measurements, and scene composition. This pattern aligns precisely with what reviewers write down: visual category-level features are described in language; identity-level features are not. The implication for product applications is that review-driven generation is a strong tool for category visualization (e.g., "show me what this kind of thing looks like") and a weak one for identity-level reproduction (e.g., "show me this specific SKU").
5. **Open-weights is closer to proprietary than expected:**
   FLUX.1-schnell tracked gpt-image-1.5 within ~3 CLIP points across all four products and was competitive on material-rich products. The proprietary model's clear advantages were brand-text rendering and unusual scene composition, both areas where larger training distributions matter

most. For projects where cost and reproducibility matter more than studio polish, the open-weights option is a defensible choice.

6. **Adaptive compute allocation is feasible without prior difficulty estimates:**
   The Controller's quadratic-surrogate-with-pseudoinverse design allocated proportionally more iterations to harder products (water bottle, jeans) than easier ones (chess set), without ever being told which products were hard. The synthetic origin point that handles the rank-deficient early-trajectory case made this work after just two real observations. This pattern generalizes to any iterative LLM pipeline where per-step quality is observable and budget is constrained.

Challenges and Lessons Learned

1. **Data acquisition required bypassing the official mirror:**
   The HuggingFace Parquet mirror of the UCSD Amazon Reviews corpus exposes only 9 of 33 categories, which would have eliminated three of our four planned products. We solved this by streaming and decompressing the gzipped JSONL files directly from mcauleylab.ucsd.edu over chunked HTTP. The lesson was practical: when a dataset is described as "available on HuggingFace," verify the coverage against the source before designing experiments around it.
2. **The visual content gate calibration was harder than expected:**
   A binary classifier that decides whether a review describes appearance has to handle reviews that mix 80% assembly notes with 20% finish observations. Early attempts to filter out reviews mentioning non-visual content (shipping, durability) over-discarded reviewers who happened to also describe appearance. The settled policy admits any review with at least one sentence of visual content, evaluated holistically per review, biased toward inclusion. The lesson is that 'visual content' is not a property of reviews but of sentences within them. The lesson is that "visual content" is not a property of reviews but of sentences within them.
3. **The Controller's design matrix was rank-deficient on early iterations:**
   A 3×3 quadratic fit needs three observations to be fully determined; with two observations, the design matrix is singular and ordinary least squares fails. We considered three options: defer surrogate fitting until enough data accumulates, fall back to a linear surrogate, or seed the system with a synthetic anchor. The third option won, because prepending a synthetic origin point (0, 0) before fitting makes the system fully determined after just two real observations and biases the early surrogate toward the natural assumption that zero refinement produces zero quality. This was the single most consequential design decision in the Controller, and it was not visible until we ran the system end-to-end.
4. **The backpack limit case was instructive precisely because it broke:**
   When the agent loop was given zero reviews to refine on, it exposed a structural property of the system: every configuration converges to the same image, and inter-model differences become entirely attributable to pre-training priors. We initially treated this as an experimental failure and considered dropping the product. We kept it because it provides the cleanest possible isolation of model-prior effects, a comparison that would have been impossible to construct deliberately.
5. **Choosing between in-loop and post-hoc metrics required separating their roles:**
   The in-loop metric drives the agent's decisions; the post-hoc metric judges the result. Conflating them, using the in-loop signal as the final score, would have hidden the jeans counter-example

entirely. The lesson is that any iterative pipeline needs at least two metric families: one the agent optimizes against, and one it has never seen.

Limitations and Future Improvements

1. **Aggregated rather than per-view ground-truth schema:**
   The ground-truth feature schema is extracted in a single GPT-4o vision call that aggregates observations across all available product photos (3–10 per product). This produces a union schema that captures any visible feature across views. A per-view schema with separate scoring per angle would surface angle-dependent failures (e.g., "the pipeline matched the front but not the back") that the aggregated schema cannot. Family B does score image-vs-image against each GT photo individually, so the aggregation issue is specific to Family C.
2. **The 13-field schema is fixed across products:**
   A schema designed for a chess set wastes fields on a backpack, and vice versa. A natural extension is a meta-prompt that proposes a product-class-specific schema before extraction, then scores against that schema. This would likely have rescued the jeans result by producing a fit-and-feel schema rather than a visual-structure schema.
3. **Surrogate functional form is fixed at quadratic:**
   The Controller assumes quality is a concave quadratic function of refinement effort. For products where the relationship is roughly linear (very easy or very hard cases), the quadratic over-fits. A model-selection step that chooses between linear, quadratic, and saturating-exponential surrogates based on AIC would adapt better to the full range of difficulty.
4. **No multimodal critic in the loop:**
   The Evaluator operates either on text-image cosine (v1, v2) or on schema-extracted features (v3). A vision-language model that compares the generated image directly to the ground-truth photo and produces structured feedback would give the Refiner a richer signal for its next rewrite.
5. **Four products is a small sample for category-level claims:**
   The 8× variation in visual pass rate is striking but rests on four data points. Extending the EDA phase to 20–30 products across more categories would let the visual-density-vs-quality relationship be characterized as a curve rather than four points.
6. **Absence of human evaluation:**
   All metrics are automated. A human-rated comparison study, where we look at shoppers that prefer the AI-generated images, and if so, for which product types, would close the loop on whether automated metrics actually predict end-user perception and usage of the tool.

Conclusion

This project asked whether AI can reconstruct a product's appearance from product metadata and customer review text alone, and the answer is conditional: yes when the review corpus is visually descriptive, and no when it is not, and the predictive variable is a property of the product category rather than the pipeline. The 8× spread in visual review density between the chess set and the backpack determined the outcome before any agent ran.

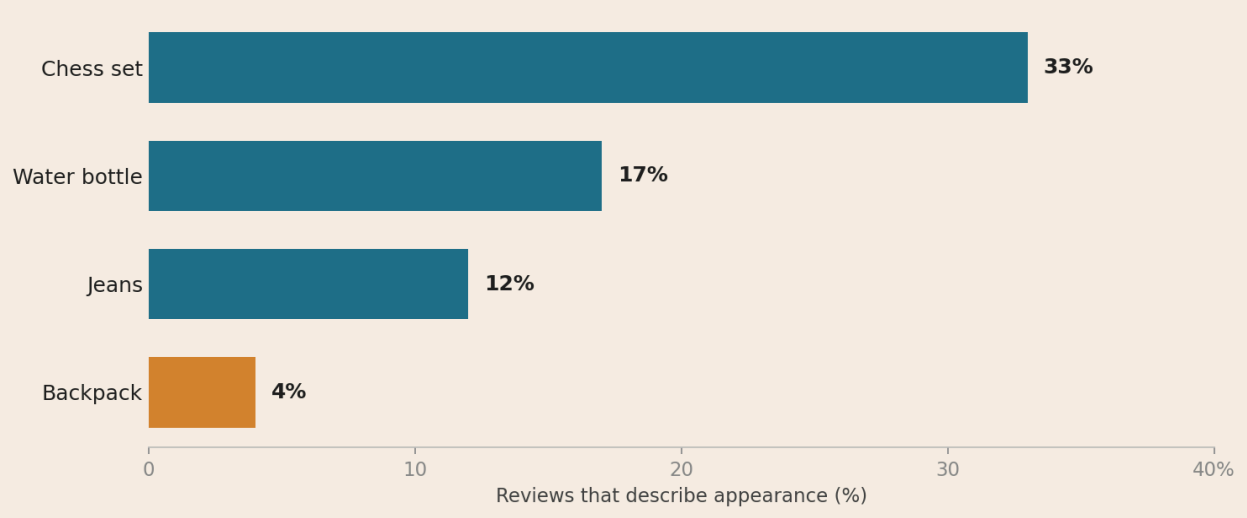
The Surrogate-Projected Adaptive Iteration Control mechanism delivered on its design goal of allocating compute proportional to observed difficulty without prior knowledge of which products were hard. Its synthetic-origin-point handling of the rank-deficient early-trajectory case is a small but consequential design pattern that generalizes to any iterative LLM pipeline operating on short trajectories.

The most analytically valuable result is the jeans counter-example, where the most sophisticated prompting strategy actively degraded output quality. This challenges the default assumption in prompt engineering that more structure produces better results and argues for matching schema design to the linguistic character of the source corpus. Together with the water bottle's metric-disagreement finding, this suggests that the dominant questions in review-driven generative pipelines are no longer "can it work?" but "for what categories?" and "measured how?"

Appendix and Testing Evidence

AI Usage
Documentation of AI usage is logged in the repo documents in AI_USE.md.

EDA Results

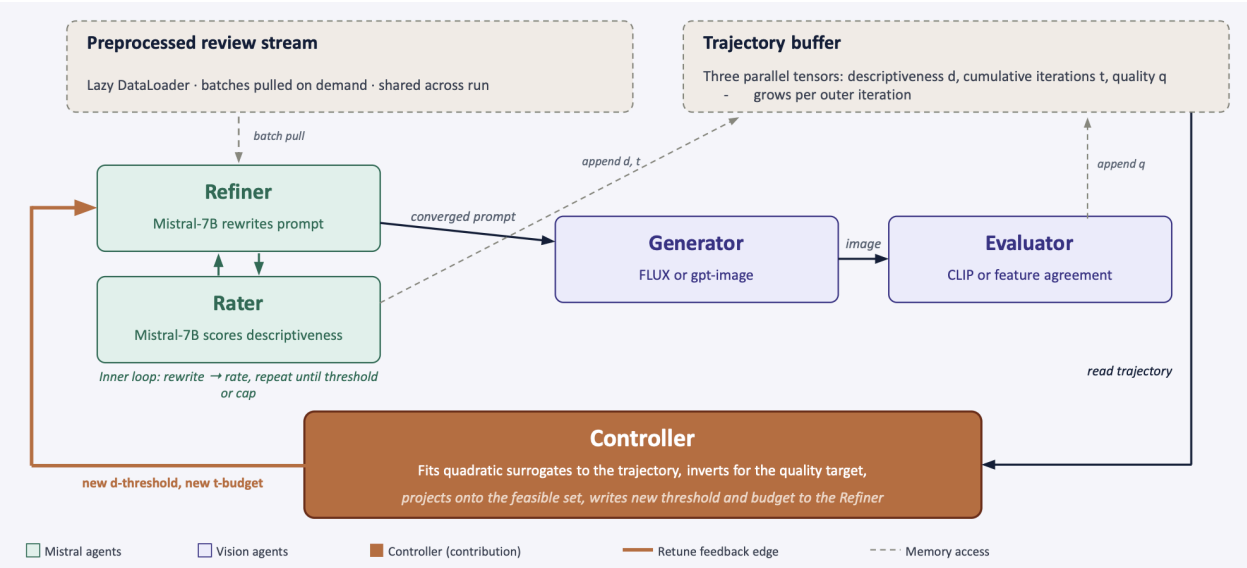


Example Structured Feature Comparison Results

field	from reviews	from ground truth
product_type	chess set	chess set
primary_colors	brown, green	brown, beige
materials	beech, birch, hornbeam, sycamore, plastic	wood
finish	matte	natural
texture	smooth	smooth
size_descriptor	large	medium
measurements	21.65 in. square	null
shape_and_form	square board, folds in half	Staunton-style pieces
decorative_elements	hand-burned flourishes, coords	carvings, inlays
visible_parts	board, pieces, clasps	pieces, board, case
brand_visibility	absent	absent
brand_description	null	null
overall_aesthetic	understated, traditional	classic, carved

Prompt Reference List

All prompts used in the canonical runs are defined as versioned, SHA-hashed string constants inside the source files that consume them. Now that experiments are complete, they have also been assembled into a reference document (PROMPTS.md) in the docs/ directory of the repo.

Prompt	Stage	Purpose	Source File	Variable Name
Visual Content Gate	Q2 preprocessing	Per-review binary classifier	src/build_filter_cache.py	LLM_SYSTEM_PROMPT_V1
Initial-Prompt Synthesis	Q2 preprocessing	Single gpt-4o call	src/generate_initial_prompt.py	INITIAL_PROMPT_SYSTEM_V1
Structured-Feature Extraction	Q2 schema / Q3 evaluation / v3 in-loop	gpt-4o vision call	src/extract_structured_features.py	EXTRACTION_PROMPT_SYSTEM_V1
Mistral Refiner	Q4 inner loop	Per-iteration prompt rewriting	src/prompt_writer.py	STEP_PROMPT_SYSTEM
Mistral Rater	Q4 inner loop	Inner-loop descriptiveness scoring	src/prompt_writer.py	RATE_PROMPT_SYSTEM

Agent Architecture Diagram